Git, SSH and the Deploy Script

What I did: I set up the Azure Ubuntu server, installed git, connected it to GitHub with an SSH key, cloned our team repository, and wrote the script that updates the server from GitHub. I also created login accounts on the server for Malcolm and Success so that they could remotely access the server to do their part of the configurations.

Step 1: Install git

sudo apt update && sudo apt install git -y

git --version

git config --global user.name "Your Name"

git config --global user.email "your GitHub email"

```
pritpal@PritpalServer:~$ sudo apt install git -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 14 not upgraded.
pritpal@PritpalServer:~$ git --version
git version 2.43.0
pritpal@PritpalServer:~$
```

```
pritpal@PritpalServer:~$ git config --global user.name Pritpal
pritpal@PritpalServer:~$ git config --global user.email Pritpal.Kaur@strathm
ore.edu
```

Step 2: Create an SSH key and connect it to GitHub

ssh-keygen -t ed25519 -C "prit-azure-server"

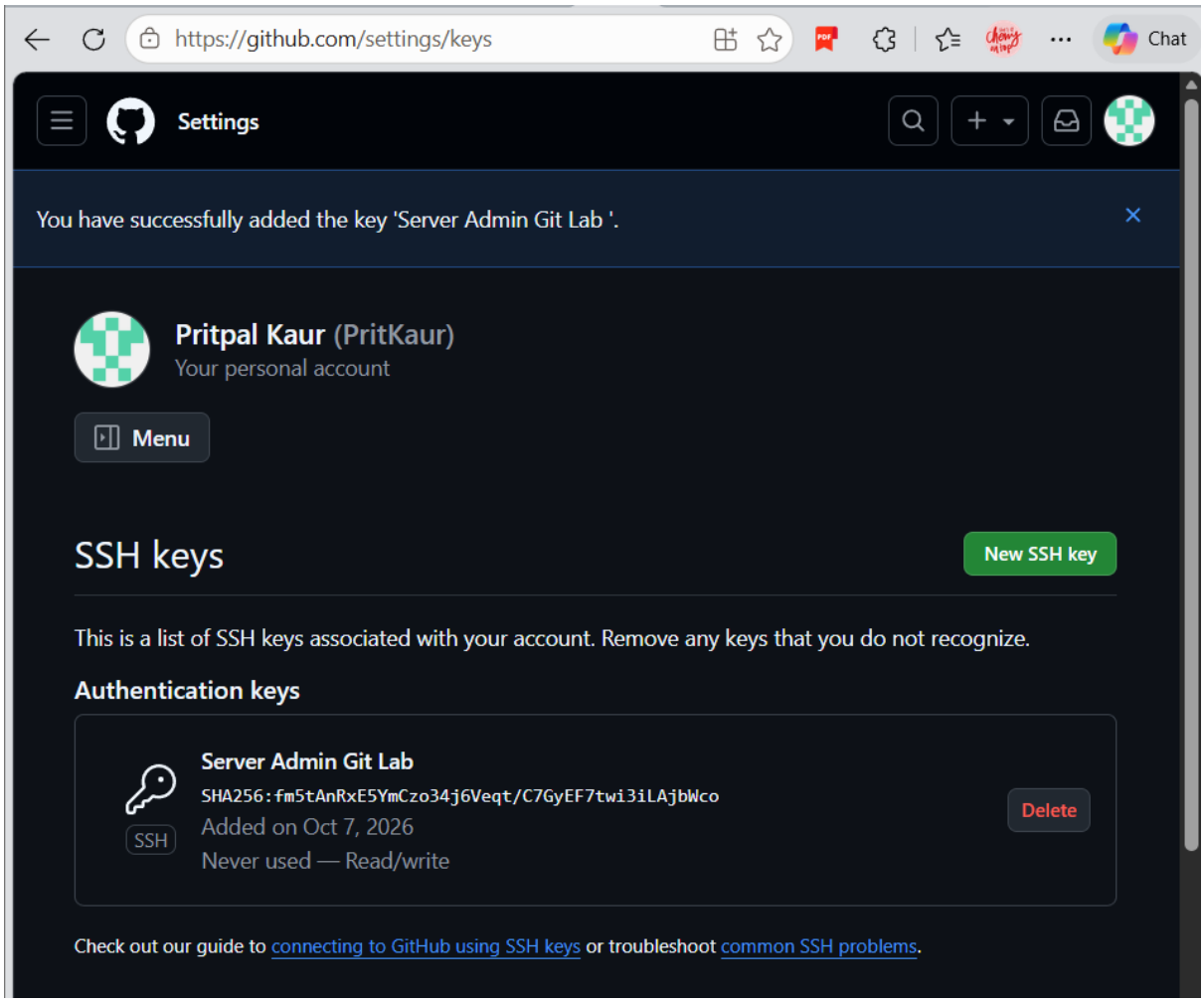
cat ~/.ssh/id_ed25519.pub

I copied the public key (the .pub file, never the private one) into **GitHub → Settings → SSH and GPG keys → New SSH key**, as an **Authentication Key**. Then I tested it:

ssh -T git@github.com

The reply was "Hi PritKaur! You've successfully authenticated, but GitHub does not provide shell access." This is normal and means that the key works.

```
pritpal@PritpalServer:~$ ssh -T git@github.com
The authenticity of host 'github.com (20.87.245.0)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOq
U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts
.
Hi PritKaur! You've successfully authenticated, but GitHub does not provide
shell access.
pritpal@PritpalServer:~$
```



Step 3: Clone the repository

After Malcolm created the repo and invited me, I accepted the invite and cloned it using the SSH address from the green **Code** button:

cd ~

git clone git@github.com:MJ416-MJ/Server-Admin-azure-lab.git

cd Server-Admin-azure-lab

git log --oneline

```
pritpal@PritpalServer:~$ git clone git@github.com:MJ416-MJ/Server-Admin-azur
e-lab.git
Cloning into 'Server-Admin-azure-lab'...
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 17 (delta 2), reused 8 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (17/17), 5.75 KiB | 5.75 MiB/s, done.
Resolving deltas: 100% (2/2), done.
```

```
pritpal@PritpalServer:~$ cd Server-Admin-azure-lab
pritpal@PritpalServer:~/Server-Admin-azure-lab$ ls
README.md  docs
pritpal@PritpalServer:~/Server-Admin-azure-lab$ git log --oneline
593de05 (HEAD -> main, origin/main, origin/HEAD) Malcolm: Updated Notes
cfdc8bd Malcolm: added guide notes file
dba3f57 Initial commit
pritpal@PritpalServer:~/Server-Admin-azure-lab$
```

Step 4: Give teammates access to the server

Malcolm and Success each made their own SSH key on their laptop and sent me only the public key. For each one I created a user and added their key to that user's authorized_keys:

sudo adduser --disabled-password --gecos "" malcolm

sudo mkdir -p /home/malcolm/.ssh

echo "THEIR_PUBLIC_KEY" | sudo tee /home/malcolm/.ssh/authorized_keys

sudo chmod 700 /home/malcolm/.ssh

sudo chmod 600 /home/malcolm/.ssh/authorized_keys

sudo chown -R malcolm:malcolm /home/malcolm/.ssh

I repeated this for success. For Malcolm's GitHub Actions workflow, I also appended his workflow's public key to my own ~/.ssh/authorized_keys, so the workflow can log in and run the deploy script.

Step 5: The deploy script

scripts/deploy.sh pulls the latest main onto the server. Each run is logged to ~/deploy.log with a timestamp, the old and new commit, and who made the latest commit. A lock stops two deploys from running at the same time.

#!/usr/bin/env bash

```
set -euo pipefail

REPO_DIR="$HOME/Server-Admin-azure-lab"
BRANCH="main"
LOG_FILE="$HOME/deploy.log"

log() { echo "$(date '+%Y-%m-%d %H:%M:%S') | $1" | tee -a "$LOG_FILE"; }
trap 'log "Deploy FAILED at line $LINENO"' ERR

exec 9>/tmp/deploy.lock
flock -n 9 || { log "Another deploy is already running, exiting"; exit 1; }

cd "$REPO_DIR"
OLD=$(git rev-parse --short HEAD)
log "Deploy started (current commit: $OLD)"

git checkout "$BRANCH"
git pull --ff-only origin "$BRANCH"

NEW=$(git rev-parse --short HEAD)
if [ "$OLD" = "$NEW" ]; then
  log "Already up to date ($NEW)"
else
  AUTHOR=$(git log -1 --pretty=format:'%an')
```

```
  MSG=$(git log -1 --pretty=format:'%s')
  log "Updated $OLD -> $NEW | last commit by $AUTHOR: $MSG"
fi

log "Deploy finished"
```

I wrote it on a branch (prit/deploy-script), pushed it, and opened pull request #1, which was merged into main. Then I tested it:

```
chmod +x scripts/deploy.sh
./scripts/deploy.sh
cat ~/deploy.log
```

The log showed the server pulling Malcolm's commits, for example Updated bf41d35 -> ebf9bd6 | last commit by Malcolm John: Merge pull request #2....